

Lecture 4: Data and Conditional Statements

Harris Coding Camp – Standard Track

Summer 2024

Today's Lecture

We often want our code to do something depending on the context.
For example,

```
if (condition is true) {  
    do this  
} else {  
    do this other thing  
}
```

We will cover:

- ▶ if else statements
- ▶ ifelse statements
- ▶ case_when() statements (later)

Conditional statements: if, else if and else

if statements

The general syntax of an if statement is as follows:

```
if (condition is TRUE) {  
  do this  
}
```

For example:

```
x <- 100  
  
if (x > 0) {  
  print("x is positive")  
}
```

```
## [1] "x is positive"
```

if else statements

Slightly more interesting, the syntax of an if else statement is:

```
if (condition is TRUE) {  
    do this  
} else {  
    do this other thing  
}
```

if else statements, example

```
x <- -5

if (x > 0) {
  print("Non-negative number")
} else {
  print("Negative number")
}

## [1] "Negative number"
```

if, else if and else statements

If we have more than 2 conditions, use if, else if and else:

```
if (condition is TRUE) {  
  do this  
} else if (second condition is TRUE) {  
  do this second thing  
} else if (third condition is TRUE) {  
  do this third thing  
} else {  
  do this other thing  
}
```

if, else if and else statements, example

```
# randomly select a number between 1 and 100  
x <- sample(1:100, 1)  
x
```

```
## [1] 55
```

```
# randomly select a number between 1 and 100  
y <- sample(1:100, 1)  
y
```

```
## [1] 45
```

```
if (x > y) {  
  print("x is greater")  
} else if (x < y) {  
  print("y is greater")  
} else {  
  print("x and y are equal")  
}
```

```
## [1] "x is greater"
```

if, else if and else can take a compound condition

```
x <- sample(1:100, 1)
```

```
x
```

```
## [1] 75
```

```
y <- sample(1:100, 1)
```

```
y
```

```
## [1] 24
```

```
z <- sample(1:100, 1)
```

```
z
```

```
## [1] 7
```

if, else if and else can take a compound condition

```
if (x > y & x > z) {  
    print("x is the greatest")  
} else if (x < y & x < z) {  
    print("x is the smallest")  
} else {  
    print("x is between y and z")  
}
```

```
## [1] "x is the greatest"
```

Try it yourself!

Let's develop a small dice game.

1. Fill in the ... so the code says "You win" if the dice add up to 7 and "You lose" otherwise.

```
dice <- sample(c(1:6), 2)

if (...) {
  print(...)
} else {
  print(...)
}
```

Hint 1: What elements are included in `dice`?

Hint 2: To sum up the values of the elements, use `sum()`.

```
sum(dice)
?sum()
```

Try it yourself!

2. Add an `else if()` block to the code above that says “try again” if the dice add up to 6 or 8.

```
dice <- sample(c(1:6), 2)

if (...) {
  print(...)
} else if (___ %in% c(6,8)) {
  print(...)
} else {
  print(...)
}
```

Hint: `%in%` is an operator used to check whether elements of one vector are present in another vector.

Conditional statements: `ifelse`

Vectorized ifelse statements

In R, the `ifelse()` function is a shorthand vectorized alternative to the standard `if else` statement.

Syntax: `ifelse(condition, returns this if condition is TRUE, returns this if condition is FALSE)`

```
x <- 5
y <- 50
ifelse(x > y, "x is greater", "x is not greater")
```

```
## [1] "x is not greater"
```

```
a <- 60
b <- 6
ifelse(a > b, "a is greater", "a is not greater")
```

```
## [1] "a is greater"
```

ifelse statements in dataframes, base R

ifelse statements work well when we need to **create a new variable**. Let's add a new variable called `in_january` to the `housing_data` based on a condition.

```
housing_data$in_january <- ifelse(housing_data$month == 1, TRUE, FALSE)
housing_data[, c("city", "month", "in_january")]
```

```
## # A tibble: 8,602 x 3
##   city      month in_january
##   <chr>    <dbl> <lgl>
## 1 Abilene      1 TRUE
## 2 Abilene      2 FALSE
## 3 Abilene      3 FALSE
## 4 Abilene      4 FALSE
## 5 Abilene      5 FALSE
## 6 Abilene      6 FALSE
## 7 Abilene      7 FALSE
## 8 Abilene      8 FALSE
## 9 Abilene      9 FALSE
## 10 Abilene     10 FALSE
## # i 8,592 more rows
```

Try it yourself

We will use `midwest` here, which is a dataset built into `tidyverse`.

```
# First, load tidyverse  
library(tidyverse)  
# Take a look at the data  
head(midwest)
```

1. Create a new variable called `poverty_designation` that is “High Poverty” if `percbelowpoverty` is above 10 and is “Low Poverty” otherwise.

```
midwest$poverty_designation <- ifelse(..., ..., ...)
```

2. Create a new variable called `ohio` that is “Ohio Counties” for observations from Ohio and “Other Midwestern Counties” for the rest of the observations.

```
midwest$ohio <- ifelse(..., ..., ...)
```

Try it yourself

3. (Optional) Create a new variable called `populous_counties` that is `TRUE` for the observations from the counties "COOK", "WAYNE", "CUYAHOGA", "OAKLAND" or "FRANKLIN" and `FALSE` otherwise. Hint: Use the `c()` and `%in%` operator.

```
# create a vector 'big_counties'  
big_counties <- c("COOK", "WAYNE", "CUYAHOGA", "OAKLAND", "FRANKLIN")  
# incorporate 'big_counties' in the following code!  
midwest$populous_counties <- ifelse(..., ..., ...)
```

4. (Optional) Create a new variable called `pop_index` that is "High" for the observations with `poptotal` greater than 100000, is "Medium" for the observations with `poptotal` between 30000 and 100000, and "Low" otherwise.

Feel free to check back in after Lecture 8!

Recap

In this lecture, we learned how to:

- ▶ use control flow with `if`, `else if` and `else` statements
- ▶ use `ifelse()` to create columns based on conditional statements

Appendix

Review: Logical AND

AND Returns TRUE when both conditions are TRUE.

##	A	B	A & B
## 1	FALSE	FALSE	FALSE
## 2	TRUE	FALSE	FALSE
## 3	FALSE	TRUE	FALSE
## 4	TRUE	TRUE	TRUE

Notice that Logical AND has a different meaning than “and” the conjunction has in common English.

Review: Logical OR

OR returns TRUE if at least one condition is TRUE.

##		A	B	A B
## 1	FALSE	FALSE	FALSE	
## 2	TRUE	FALSE	TRUE	
## 3	FALSE	TRUE	TRUE	
## 4	TRUE	TRUE	TRUE	

Notice that Logical OR has a different meaning than “or” the conjunction has in common English.

Review: Logical OR

What is the output?

`(5 > 7) | (10 == 10)`

Review: Logical OR

Recall that `==` is the logical comparison for if two things are equal.

```
# 5 is greater than 7 OR 10 equals 10"  
(5 > 7) | (10 == 10)
```

```
FALSE | TRUE
```

```
## [1] TRUE
```

Review: Logical NOT

Let's see how NOT (!) works.

```
!TRUE
```

```
## [1] FALSE
```

```
!FALSE
```

```
## [1] TRUE
```

Review: Logical NOT

With conditional statement(s):

```
!(2 > 1)
```

```
!TRUE
```

```
## [1] FALSE
```

ifelse statements in dataframes for multiple categories

If we have many categories, use **nested** ifelse statement.

Say we want to add a new variable called `median_ref` which value can be High, Medium or Low:

- ▶ If median is at least 70k: High
- ▶ If median is between 60k and 70k: Medium
- ▶ If median is lower than 60k: Low

```
housing_data$median_ref <- ifelse(housing_data$median >= 70000, "High",  
                                  ifelse(housing_data$median < 70000 &  
                                          housing_data$median >= 60000,  
                                          "Medium", "Low"))  
housing_data[, c("city", "median", "median_ref")]
```

```
## # A tibble: 8,602 x 3  
##   city      median median_ref  
##   <chr>    <dbl> <chr>  
## 1 Abilene  71400 High  
## 2 Abilene  58700 Low  
## 3 Abilene  58100 Low  
## 4 Abilene  68600 Medium  
## 5 Abilene  67300 Medium  
## 6 Abilene  66900 Medium
```